AWS Cloud Services

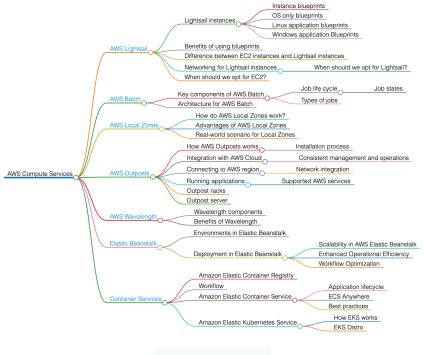
Thursday, 25 June 2026 11:44 AM

Introduction to AWS Compute Services

Explore the fundamentals of AWS compute services to understand how they provide processing resources like CPU and RAM. Learn about services such as AWS Lightsail, Batch, Local Zones, Outposts, Wavelength, Elastic Beanstalk, and container options to effectively manage and deploy cloud applications.

Compute services is an Infrastructure-as-a-Service (IaaS) model that offers processing resources for multiple purposes. Compute resources include CPU, RAM, and network cards for general, high-speed graphic processing, or high-performance computing. Consumers rely on cloud providers to manage underlying hardware to cater to their needs.

AWS provides a wide range of compute services tailored to user requirements. In this module, we will explore several services enabling users to meet their computing needs effectively.



Amazon Lightsail

Explore Amazon Lightsail, an accessible AWS compute service designed for quick VPS deployment. Understand its instance and blueprint options, networking features, and how it compares with EC2 for various use cases.

Amazon Lightsail is a simplified, low-cost compute service that Amazon Web Services (AWS) provides. Lightsail provides all the essential tools for launching a project quickly. This includes instances, containers, databases, networking, storage, DNS management for registered domains, and resource backups.

Amazon Lightsail offers a user-friendly interface that simplifies the process of deploying and managing websites, applications, and other cloud-based workloads.

Lightsail instances

Lightsail instances are virtual private servers (VPS) that we launch in the cloud. They provide computing power, memory, and storage resources. Instances are among the core computing services offered by AWS. Lightsail gives us the flexibility to select an image that includes an operating system, along with images that come with preconfigured applications or development stacks and the base operating system.

Instance blueprints

Instance blueprints in Amazon Lightsail are pre-configured templates that can be used to launch virtual private servers (VPS) with specific software stacks or applications already installed. Blueprints help simplify the process of setting up and deploying servers by providing ready-to-use configurations tailored for common use cases.

Lightsail offers various blueprints for different operating systems and applications. These blueprints fall into two main categories: **OS Only** and **Applications**.

OS Only blueprints

These blueprints provide a bare operating system environment without any additional software installed. They are ideal for users who want to set up their environment from scratch. Lightsail provides us with Linux and Windows distributions.

Application blueprints

These blueprints come with pre-installed software or application stacks, making it easier to deploy common applications quickly. Lightsail provides us with Linux application blueprints as well as Windows application blueprints.

Benefits of using blueprints

Here are some of the benefits of using blueprints:

- Blueprints significantly reduce the time required to set up an instance with a specific software stack. We can launch a fully configured server in just a few clicks.
- Blueprints ensure that the software is installed and configured consistently across multiple instances, reducing the risk of configuration errors.
- Blueprints simplify the process for users who may not be familiar with server setup and software installation. They provide a ready-to-use environment tailored to specific needs.
- Blueprints provide a predefined setup, but users can still customize the instances after launch to meet their specific requirements.

Networking for Lightsail instances

Amazon Lightsail instances are hosted within an AWS Virtual Private Cloud (VPC). Each AWS account has a default VPC that AWS manages specifically for Lightsail instances. This VPC is separate from the default VPC used by other AWS services, such as EC2. Lightsail abstracts much of the VPC networking complexity, providing a simplified experience for users. When a Lightsail instance is created, it automatically resides within this default VPC without requiring manual VPC setup.

Lightsail instances use security groups to control inbound and outbound traffic. By default, each Lightsail instance is assigned a security group with rules that can be modified to allow or restrict traffic based on our needs.

Each Lightsail instance operates within its own private network inside the AWS-managed VPC, ensuring network isolation and security. The private IP address assigned to the instance is only accessible within the VPC, while public IP addresses can be assigned to allow external access.

EC2 instances vs. Lightsail instances

Let's look at some of the use cases when Lightsail and EC2 ae most suited:

When should we opt for Lightsail?

Some of the use cases where Lightsail is a better choice are:

- **Simple web hosting**: Lightsail is suitable for hosting small websites, blogs, or personal projects as it provides easy-to-use blueprints and a simplified management interface, making it ideal for users who need to get online quickly without extensive cloud knowledge.
- **Small business applications**: Lightsail is suitable for running business applications such as CRM systems, project management tools, or small e-commerce sites. Its fixed pricing and simplified management are beneficial for small businesses with limited IT resources.
- **Static websites and blogs**: Lightsail is suitable for hosting static websites or content-heavy blogs as Lightsail offers pre-configured application stacks that simplify the deployment of content-focused websites.

When should we opt for EC2?

- **High-performance computing:** EC2 is suitable for running applications that require significant computational power, such as scientific simulations, big data processing, or machine learning. EC2 provides a diverse selection of instance types tailored for various workload requirements, including instances with GPUs and high CPU/RAM configurations.
- **Scalable web applications:** EC2 is suitable for building web applications that need to scale dynamically based on traffic. We can use EC2 Auto Scaling to handle varying web traffic for an e-commerce site. EC2's auto-scaling and load-balancing features provide robust scalability options.
- **Complex and custom architectures:** EC2 is suitable for deploying applications with complex networking, security, and architectural requirements. It is also more suitable for building a multi-tier web application with separate web, application, and database layers because EC2 allows for detailed configuration of VPCs, security groups, and IAM roles, which enables fine-grained control over the infrastructure.
- **Enterprise applications:** EC2 is suitable for running enterprise applications like SAP, Oracle, or custom enterprise software. Its flexibility and integration with other AWS services make it suitable for enterprise-level applications that require high availability and performance.

	EC2	Lightsail instance
Features	Provides a wide range of instance types, storage options, and networking features. Offers granular control over the underlying infrastructure.	Offers pre-configured development stacks and applications. Limited in terms of customization and advanced features.
Pricing	More complex pricing based on usage (compute, storage, network), which can potentially be optimized using various pricing models	Predictable and simple pricing model with flat-rate monthly fees.
Networking	Advanced networking capabilities, including detailed control over VPC, security groups, network ACLs, and direct integration with other AWS services.	Simplified networking setup with basic options.
Integration with AWS Services	Limited integration with other AWS services.	Deep integration with a broad range of AWS services, providing extensive capabilities for building and scaling applications.

AWS Batch

Explore how AWS Batch simplifies running batch computing workloads by managing job scheduling, compute resource allocation, and execution. Understand key components like job definitions, job queues, and compute environments to optimize batch job processing and cost efficiency on AWS.

AWS Batch is a fully managed service designed for running batch computing workloads on the AWS Cloud. It handles high-volume computing tasks by breaking them into smaller jobs that can be processed concurrently. AWS Batch automatically allocates the ideal amount and type of compute resources, such as CPU or memory-optimized instances, based on the needs of the submitted batch jobs.

AWS Batch effectively organizes, schedules, and executes batch workloads using various AWS resources such as Amazon ECS (Elastic Container Service), EKS (Elastic Kubernetes Service), and AWS Fargate. It offers the flexibility to use spot instances for cost optimization.

Key components of AWS Batch

The following are the key components of AWS Batch:

- **Job:** It is a unit of work that you submit to the service to be run. Jobs are the fundamental building blocks of AWS Batch and can consist of a single command or script or an entire workflow of commands and scripts.

script or an entire workflow of commands and scripts.

- **Job definition:** This specifies how a job has to run. It includes details such as the Docker image to use, the required resources (like vCPUs and memory), environment variables, and IAM roles. Job definitions are reusable and can be referenced when submitting jobs to AWS Batch. The parameters specified in the job can be modified at runtime.
- **Job queues:** Jobs are submitted to job queues, waiting to be scheduled and run. AWS Batch uses job queues to manage job priorities and dependencies. We can define multiple job queues with different priority levels to manage the jobs' execution order. A job remains in the queue until it is assigned to a compute environment. We can link one or more compute environments to a job queue.
- **Compute environment:** Compute environment consists of a collection of managed or unmanaged compute resources (e.g., Amazon EC2 instances, Spot instances) used to run jobs. Managed compute environments are provisioned and scaled by AWS Batch, while the user manages unmanaged compute environments.

Job lifecycle

The AWS Batch job lifecycle includes the following phases:

1. **Submission:** After a job definition is registered, the job can be submitted to the AWS Batch job queue. We can override the parameters specified in the job definition at runtime.
2. **Scheduling:** AWS Batch scheduler evaluates the job's requirements and priorities and assigns it to a compute environment with the necessary resources.
3. **Execution:** The job runs as a containerized application on the assigned compute resource. AWS Batch orchestrates starting, stopping, and scaling the underlying compute resources as needed.
4. **Completion:** After completing the job, its status is updated in the job queue. We can track job status and progress using the AWS Management Console, AWS CLI, or AWS SDKs.

AWS Local Zones

Explore how AWS Local Zones extend AWS Regions by placing compute and storage resources closer to users for single-digit millisecond latency. Understand their role in hybrid cloud setups, how to configure Local Zones using VPC subnets, and real-world applications like Netflix's content creation. Gain insight into benefits such as improved performance for latency-sensitive workloads and options for disaster recovery through geographic distribution.

AWS Local Zones is an AWS infrastructure deployment that places AWS infrastructure closer to densely populated areas, industries, and IT hubs. Local Zones enable us to run applications that require single-digit millisecond latencies to end-users or on-premises installations. Each Local Zone has its own internet connection and supports AWS Direct Connect, making it possible to serve applications that demand low-latency connectivity.

How do AWS Local Zones work?

Configuring Local Zones involves creating a Virtual Private Cloud (VPC) in our parent AWS Region and extending it to include subnets in Local Zones. These subnets become part of our VPC, allowing us to deploy resources such as EC2 instances and EBS volumes for low-latency access. We manage resources in Local Zones through the same AWS Management Console, CLI, and SDKs used for our overall AWS infrastructure, ensuring a seamless management experience. Local Zones are connected to the parent Region via high-bandwidth, secure network links, ensuring low-latency communication between resources.
The illustration below shows an account with a VPC in `us-east-1` that is extended to the Local Zone `us-east-1-dfw-1a`. In the VPC, every zone contains a single subnet, and each subnet hosts one EC2 instance.

Working of AWS Local Zones

Advantages of AWS Local Zones

Here are some of the benefits of AWS Local Zones:

- **Low latency:** It provides low-latency access to AWS services for geographically specific locations, which benefits applications requiring real-time responses, such as gaming, media & entertainment, and machine learning.
- **Extension of AWS regions:** AWS Local Zones are extensions of AWS Regions. They allow us to run some of our latency-sensitive applications close to end users while seamlessly connecting to the rest of our applications running in the AWS Region.
- **Other AWS services deployment:** Local Zones offer various AWS services, including compute (Amazon EC2), storage (Amazon EBS), and database services, which enable us to build and deploy applications closer to our end-users.
- **Hybrid deployments:** They support hybrid cloud deployments, where part of the infrastructure is on-premises and a part is in the AWS Cloud, providing consistent and low-latency network connectivity between them.

Real-world scenario for Local Zones

In 2019, Netflix launched its VFX studio on NICE DCV, a high-performance remote display protocol, and powered its remote workstations with Amazon EC2 for secure, resizable compute capacity. To further reduce application latency and ensure smooth remote operations, Netflix began using AWS Local Zones in 2020. This setup allowed Netflix to handle latency-sensitive workloads closer to the artists, ensuring a smooth and efficient content creation experience.

Netflix uses high-performance EC2 resources on Local Zones to boost performance for graphics-intensive applications like video rendering. This upgrade enabled Netflix to provide high-performance virtual workstations, significantly enhancing the content creation capabilities for its artists. Additionally, by using Local Zones, Netflix achieved not only decreased latency but also increased availability for its applications, leveraging the full suite of AWS services and infrastructure for high availability.
Using AWS Local Zones can improve performance for low latency applications and provide additional disaster recovery options by geographically distributed

infrastructure.

AWS Outposts

Explore AWS Outposts to understand how it enables hybrid cloud solutions by extending AWS compute, storage, and database services on-premises. Discover how to run low-latency applications close to end-users or local resources while maintaining integration with AWS cloud for seamless management and scalability.

AWS Outposts is a fully managed service that extends AWS infrastructure, services, APIs, and tools to virtually any data center, co-location space, or on-premises facility. This service enables customers to run AWS services on-site, ensuring a consistent hybrid experience. AWS Outposts extends native AWS services, infrastructure, and operating models to on-premises locations, enabling low-latency access to applications and providing a consistent experience across both cloud and on-premises environments.

Outposts are ideal for low-latency applications that must be close to on-premises resources or end-users, local data processing and storage to meet data residency requirements, and applications that must stay on-premises due to latency sensitivity, data residency rules, or legacy systems. They are also perfect for hybrid cloud deployments where applications must integrate with both on-premises systems and AWS cloud resources.

How AWS Outposts works

Customers can select from various fully integrated and pre-validated Outpost configurations tailored to specific use cases and facility requirements. They can choose from different EC2 instance types, with options for local storage and EBS SSD general-purpose volumes, to meet their specific application needs. Once ordered, the Outpost hardware is delivered and installed at the customer's location by AWS personnel.
Outposts are fully managed and maintained by AWS, ensuring that software and firmware updates are automatically applied. Outposts integrate seamlessly with the broader AWS cloud, enabling hybrid cloud deployments. Applications can easily interact with other AWS services in the cloud, such as S3, DynamoDB, or machine learning services. This integration allows for consistent operations, APIs, and management tools across both on-premises and cloud environments.

Supported AWS services

- **Compute:** Run EC2 instances on Outposts for various computing needs.
- **Storage:** Utilize EBS volumes for block storage and access S3 for object storage.
- **Databases:** Deploy RDS instances locally or use DynamoDB for low-latency access to NoSQL databases.
- **Machine learning:** Integrate with AWS machine learning services to process data locally and in the cloud.

Connecting to AWS region

Customers connect their Outpost to the nearest AWS region using AWS Direct Connect or VPN. This ensures low-latency access to both on-premises and cloud resources. A local gateway can be created to connect the Outpost to local networks, facilitating seamless integration with existing on-premises infrastructure.
Customers can expand their VPC by setting up a subnet linked to their Outpost. This enables instances on the Outpost to securely interact with other instances in the VPC using private IP addresses.

Network integration

Let's consider a scenario where we want to connect the local network with AWS resources using AWS Outposts. We can extend our VPC by creating subnets and associating them with the Outposts, allowing instances to communicate securely using a Direct Connect or a VPN.

AWS Outposts servers and racks

An **Outpost Server** is a component of AWS Outposts, designed to provide a smaller, more flexible option for deploying AWS infrastructure on-premises. Outpost Servers offer a more granular and flexible deployment option. Outpost servers are suitable for smaller-scale deployments and cost-effective solutions.
The Outpost rack, on the other hand, is a fully managed rack of servers, networking gear, and other infrastructure components designed for installation in a data center or co-location space. Outpost racks are suitable for large-scale deployments and complex workloads.

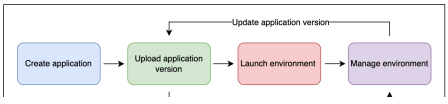
AWS Elastic Beanstalk

Explore how to deploy and manage applications on AWS Elastic Beanstalk. Understand environment tiers, deployment methods, and features like automatic scaling and CI/CD integration to optimize your cloud application workflow.

AWS Elastic Beanstalk allows the quick deployment and management of applications in the AWS Cloud. We simply need to upload our application on Elastic Beanstalk, which provides and manages the resources required to deploy our application in the cloud. Elastic Beanstalk can deploy applications developed in Go, Java, .NET, Node.js, PHP, Python, and Ruby.

How AWS Elastic Beanstalk works

To deploy an application using Elastic Beanstalk, first, we need to create an application and upload its source code on Elastic Beanstalk. This launches an environment for us and configures the resources required to deploy our application. Once the environment is launched, we can manage it and deploy new versions of our application. We can see the flow of Elastic Beanstalk in the diagram given below:



Elastic Beanstalk workflow

Environments in Elastic Beanstalk

In AWS Elastic Beanstalk, an environment is a collection of resources used to deploy a specific version of our application. A single application can have multiple environments, each used to host a specific version of an application. When launching an environment in Elastic Beanstalk, we first need to select one of the two environment tiers:

- **Web server tier:** In the web server tier, we usually deploy web applications that receive HTTP requests from the clients, such as APIs and other interactive applications. Load balancers are launched in this tier to control and distribute incoming traffic.
- **Worker tier:** The worker tier doesn't include a load balancer and is typically used when we need to execute lengthy computational tasks, process data, or perform other backend operations that don't require direct contact with the client. Applications deployed in a worker tier usually use SQS queues to fetch data and other requests.

Deployment in Elastic Beanstalk

AWS Elastic Beanstalk provides us with various deployment methods we can choose from depending on our application's requirements:

- **All at once:** This is the quickest deployment method for deploying a new version of our application simultaneously across all instances. During deployment, all instances are temporarily out of service.
- **Rolling and Rolling with additional batch:** In these methods, our application is deployed in batches to avoid downtime and increase availability.
- **Immutable:** In this method, Elastic Beanstalk makes sure our application is always launched on a new instance, making it a relatively slow method. One advantage of this method is that it provides a quick rollback in case our deployment fails due to any reason.
- **Traffic splitting:** In this method, we can test the health of the new version of our application by directing a part of the incoming traffic to a new version and directing the rest of the traffic to the old version of our application.
- **Blue/Green**: In this method, a new version of our application is deployed in a new environment. We can then swap the canonical names of the two environments to ensure our application doesn't become unavailable to the user.

Features of AWS Elastic Beanstalk

Now, let's look at some of the features of AWS Elastic Beanstalk:

- **Scalability:** Elastic Beanstalk manages scalability by automatically adjusting compute resources according to application needs. It handles the number of Amazon EC2 instances required without manual intervention, making it ideal for applications with fluctuating workloads. During traffic spikes, Elastic Beanstalk can scale up resources to meet increased demand and scale down when traffic decreases, ensuring both performance and cost efficiency.
- **Enhanced operational efficiency:** Elastic Beanstalk simplifies system administration by automating tasks such as operating system patching and updating application components like web servers and programming languages. This managed service ensures that the application environment is secure, stable, and current, allowing developers to focus more on application development rather than maintenance.
- **Workflow optimization:** Elastic Beanstalk enhances developer productivity with its integration into development tools. It supports direct deployments from platforms like Git, Eclipse, and Visual Studio, facilitating a streamlined continuous integration and continuous deployment (CI/CD) pipeline. This integration reduces deployment friction, enabling faster updates and improvements, and boosting overall productivity.

Container Services

Explore AWS container services such as Elastic Container Service (ECS), Elastic Kubernetes Service (EKS), and Elastic Container Registry (ECR). Understand the distinctions between containers and virtual machines, and learn best practices for deploying and managing containerized applications on AWS.

Containers and virtual machines (VMs) are both essential technologies for creating isolated environments to run applications, but they differ fundamentally in their architecture and resource utilization.

- **Containers** are lightweight units that package an application and its dependencies, sharing the host operating system's kernel, which makes them highly efficient and quick to start. This architecture allows containers to be portable and consistent across various environments, ideal for microservices and DevOps practices.
- In contrast, **VMs** run on a hypervisor and include a complete guest operating system along with the application, providing stronger isolation at the cost of increased resource consumption and slower startup times. VMs are suited for scenarios requiring robust isolation and the ability to run different operating systems on the same hardware. These differences make containers a preferred choice for modern, agile development practices, while VMs remain valuable for applications needing more stringent isolation and OS-level control.

Virtualisation vs Containerisation

Docker is the most popular platform for simplifying container development, shipping, and management. Docker separates the application from the underlying infrastructure so that applications can run quickly on different platforms, regardless of hardware.

Amazon Elastic Container Registry

Amazon Elastic Container Registry (ECR) is an AWS-managed Docker container registry service. It provides a secure and scalable repository for storing, managing, and deploying Docker images. ECR offers both public and private registries that can allow multiple repositories.

ECR is primarily designed as a private registry, meaning that it's intended for use within organizations or teams. A **public registry** is a container registry publicly accessible to anyone on the internet. It hosts a wide range of open-source and community-contributed Docker images, making them readily available for

developers to pull and use in their projects.

Amazon Elastic Container Service

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service by AWS. It simplifies the deployment and management of containerized applications, allowing us to run Docker containers at scale. ECS integrates seamlessly with other AWS services, providing a flexible and efficient solution for deploying and managing containerized workloads.

ECS Anywhere

ECS Anywhere by Amazon ECS is container orchestration software to manage containers on-premises. ECS Anywhere eliminates the need for in-house container orchestration. It extends ECS to the on-premises servers or VMs, offering a unified way to run and manage containerized workloads across all your environments — cloud and on-premises—with a familiar ECS experience.

Best practices

Let's take a look at some of the best practices for ECS.

- It's important to note the price of Fargate; Fargate is usually more expensive than the EC2 instance for the application running 24/7, EC2 is a much better choice than the Fargate.
- It is important to understand the security concerns for the containers running applications in the case of EC2 as the underlying infrastructure; it's important to keep an eye on the Security patches of the OS and update your images based on that.
- In the case of EC2 as an underlying infrastructure, different containers consume different resources, such as CPU or memory. Use CloudWatch events and tag the containers to efficiently monitor the resources.

Amazon Elastic Kubernetes Service

Amazon Elastic Kubernetes Service (EKS) is a fully managed and certified service that simplifies managing Kubernetes clusters on AWS. EKS simplifies containerized applications' deployment, management, and scaling using Kubernetes on AWS infrastructure. EKS offers different compute services, from serverless Fargate to EC2 to offer high availability, security, and seamless integration with other AWS services for enhanced development and operational efficiency.
EKS is versatile and adaptable to various use cases, offering developers and organizations the flexibility, scalability, and efficiency required to modernize their applications. EKS is well-suited for deploying web applications, including e-commerce platforms, content management systems, and media streaming services. It efficiently scales resources to handle fluctuating traffic loads while maintaining high availability and performance.

EKS Distro

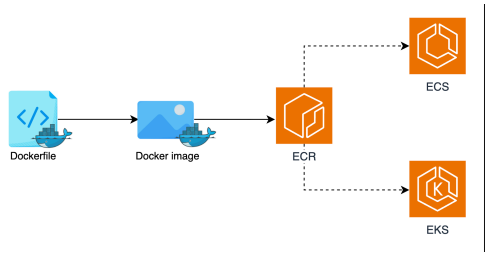
Amazon EKS Distro (EKS-D) is an open-source distribution of Kubernetes provided by AWS. It offers the same core components that power Amazon Elastic Kubernetes Service (EKS) for creating and managing containerized applications. The difference between EKS and EKS-D is that the latter is available to install and manage locally. EKS-D can be used on-premises, in a cloud, or in any system. EKS-D provides a path to having essentially the same Amazon EKS Kubernetes distribution running wherever you need to run it.

EKS Distro automatically creates and manages Kubernetes clusters. It offers a consistent Kubernetes experience across AWS cloud and data centers. It alleviates the need to manually track, update, and determine Kubernetes compatibility across different teams. EKS distro also offers installable, reproducible Kubernetes builds for cluster creation as well as security patches even after the community support expires.

Workflow

The workflow to deploy containers over AWS is as follows:

- **Build and configure a Container**: Begin by creating a container image that includes all necessary components for the application to run.
- **Store the Container image in Amazon ECR**: Next, create an ECR repository to store the container images.
- **Deploy and Manage Containers with Amazon ECS or EKS**: Finally, deploy the container images to ECS or EKS, which will manage the deployment and scaling of the containers.



ntroduction to Amazon EC2

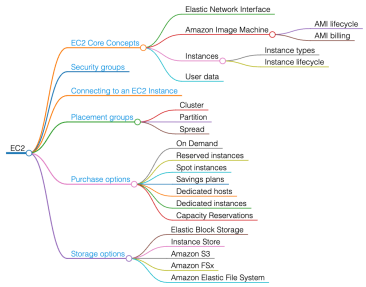
Explore Amazon EC2 to understand how it provides resizable compute capacity for running virtual servers. Learn its features like AMIs, instance types, storage options, and security to manage cloud resources efficiently and cost-effectively.

Cloud computing has become a significant component in the digital landscape due to its flexibility and cost efficiency. Organizations and businesses have migrated from on-premises architecture to the cloud due to higher costs and a rigid structure. Organizations had to manage, secure, and maintain servers; this became a hassle and a costly solution. The running cost of on-premise servers was a big problem in terms of scalability. Cloud providers, like AWS, came up with a scalable and cost-effective solution, Elastic Cloud Computing, famously known as EC2.

Elastic Compute Cloud (EC2)

Amazon Elastic Compute Cloud (EC2) offers resizable compute capacity in the cloud to flexibly scale the resources up and down based on the demand. It allows users to run virtual servers, known as instances, for various computing tasks. EC2 offers a broad set of servers with the latest processors, storage, networking, operating systems, and different purchase models, including pay-as-you-go. EC2 is a secure, flexible, and scalable solution, enabling businesses to easily deploy, manage, and scale applications without investing in physical hardware.

In this chapter, we will cover in detail the different features of EC2: AMIs, instances, instance types, storage, and security options. We will learn about the user data scripts to launch EC2 in dynamic and scalable environments. We will explore how each feature affects the overall cost of the service. We will also learn how EC2 allows for the control of the deployment of instances using placement groups.



Outline of the chapter

EC2 Core Concepts

Explore the fundamental concepts of Amazon EC2 including the Elastic Network Interface for networking, Amazon Machine Images for launching instances, and the variety of EC2 instance types suited for different workloads. Learn about the lifecycle states of instances like launch, stop, and termination along with how to automate tasks using EC2 user data scripts, giving you a solid foundation for managing AWS compute resources.

In this lesson, we will go over the Elastic Network Interface, Amazon Machine Image (AMI), the different types of instances offered by EC2 and their use cases, and EC2 user data.

Elastic Network Interface (ENI)

An **Elastic Network Interface (ENI)**, also known as a Network Interface, is a virtual network card that can be attached to the EC2 instances. ENI is a logical networking component that can be used to provide multiple IP addresses or attach an instance to different subnets. An Elastic Network Interface (ENI) in AWS is associated with an Availability Zone (AZ) in a region. This means that we can attach and detach the ENI to different instances within the same Availability Zone.

Every instance launched has a default network interface, known as the primary network interface; by default, it offers a private IP address to the instance. However, it can be configured to offer the public as well as the elastic IP address. A primary network interface can not be detached from an instance. However, we can attach more network interfaces to an instance. The number of network interfaces that can be attached to an instance depends upon the instance type and size. For example, `m1.xlarge` can have up to 4 network interfaces; similarly, `t2.micro` can have 2 network interfaces maximum.Types of IP addresses
ENI offers different types of IP addresses, each offering a unique set of characteristics. The three different types of IP addresses are: Public, Private, and Elastic IP address.

- **Public IP** addresses are used to communicate over the internet. AWS allows us to control whether an instance in the network receives a Public IP address or not. Sometimes, we do not want our instance to communicate with the internet directly to make it more secure. The public IP address of instances remains associated with the instance until it’s stopped or terminated.
- **Private IP** addresses are essential for communicating within a VPC. Each instance in the network has a unique private IP address that is used as the identifier for that instance.
- **Elastic IP** address offers a static public IP that can be attached directly to an EC2 instance or a network interface. The elastic IP address remains associated until removed and attached to a different instance or a network interface.

Amazon Machine Image (AMI)

An **Amazon Machine Image (AMI)** is a pre-configured image provided by AWS that contains the necessary information required to launch an instance. AMI serves as a template for the root volume that includes information regarding the operating system, application server, and applications.

An AMI must be specified to launch an instance; multiple instances can be launched from a single AMI. We can also create our AMIs, copy an AMI from one region

to another, and deregister an AMI when it is no longer needed.

We can select an AMI based on the following characteristics:

- **Operating system:** AWS offers different AMIs based on different operating systems, such as Amazon Linux 2 AMI and Microsoft Windows Server 2019 Base AMI.
- **Regions:** AWS offers different AMIs based on the region; AMIs can be transferred from one region to another. It is important to note that transfer between distant regions would take more time to launch the EC2 instance.
- **Architecture (32-bit or 64-bit):** AWS offers various AMIs for different types of architecture, such as Ubuntu 20.40 LTS for 64-bit architecture and Amazon Linux for 32-bit architecture. It is important to note that AWS also offers Amazon Linux AMI for 64-bit architecture.
- **Launch permissions:** Each AMI has predefined launch permissions set by its owner. It is used to determine the availability of that AMI. There are three types of launch permissions:
- **Public:** Available to all the AWS accounts
- **Explicit:** Available to specific AWS accounts, organizations, or organization units
- **Implicit:** Available only to the owner
- **Storage for the root device:** AMIs are also categorized based on the root storage for the instance. Each instance either has an Amazon EBS or an Amazon instance store as a root device.

AMI billing

AWS offers multiple AMIs to launch different instances. These AMIs support various operating system architectures and offer different features. It is important to understand their effect on the AWS bill. AMI cost depends upon the OS and the different features offered by the respective AMI. It is important to understand the cost of AMIs before launching instances.

Instances

An instance is a virtual server configured using an AMI used to launch the instance. It can run different operating systems, including Linux, Windows, and macOS. AWS offers different types of instances based on the computing power or amount of memory.

Instance types

Instances are categorized based on their computing power, memory, and networking capabilities. When launching an instance, the instance type specifies the hardware of the server. Let's look at the different types of instances below:

- **General purpose:** These instances offer a balance of compute, memory, and networking capabilities that can be used for a wide range of workloads. This also includes Mac instances that are useful for building, developing, and testing Apple applications.
- **Compute optimized:** These instances offer high compute power, making them ideal for intense applications that require intense processing.
- **Memory optimized:** These instances offer fast memory performance, specially designed for workloads that require large datasets in memory.
- **Storage optimized:** These instances are designed to offer high sequential read and write capabilities on local storage.
- **Accelerated computing:** These instances use hardware accelerators to offer complex calculations in a more efficient manner. They offer more parallelism for intensive workloads.
- **High-performance computing:** These instances are built to offer the best price-performance for high-performing compute instances. They are normally used to solve complex computational problems.

It is important to identify different types of instances from each other. Instances types are named after family, generation, processor family, additional capabilities, and size. Let's understand the nomenclature of instance types with an example. Consider an instance `r7gd.16xlarge`:

- The first position in the instance type name is used to refer to the instance family.
- The second position is used to represent the instance generation.
- The third position is used to represent the processor family in the instance.
- The last position before the period (`.`) i.e., the fourth position is used to highlight additional capabilities of the instance. After the period (`.`), the instance size is represented such as `small`, `xlarge` and etc.

Nomenclature of instance type

Each instance has a root volume attached to boot the instance. After launching, an instance works similarly to a server, and it keeps running until stopped, hibernated, terminated, or failed.

Instance lifecycle

The lifecycle of EC2 transits through different states from creation to termination. It's important to understand the behavior of instances to fully understand how EC2 works. An instance is a server in the cloud; naturally, it can be launched and terminated like a local server. However, EC2 instances can also be stopped, hibernated, rebooted, and retired. Let's take a deeper look at the different states of EC2 instances.

- **Launch:** When an instance is launched from a selected instance type, its state is updated from pending to running.
- **Stop:** Once the instance is running, we can stop and start the instance again anytime. It is important to note when an instance is stopped, the data stored in the RAM is lost, and the instance may not have the same public IP address, but the private IP stays the same. The instance store volumes are also lost; AWS offers an alternative to preserve the data stored in the RAM using Hibernation.
- **Hibernation** saves the content of RAM to the EBS root volume. It also preserves the attached EBS volumes. During reboot, the contents of RAM saved earlier are

again loaded into the memory.

- **Termination:** Once the work is done and we no longer require the instance, we can terminate the instance. Terminating an instance changes the state of the instance from “Running” to “Shutting down.” The instance can not be started again after it has been terminated.

EC2 user data

User data allows to run commands/scripts when launching an EC2 instance to automate configuration tasks and even run scripts after the instance starts. It can be a script or cloud-init directives; scripts can be a shell script or any other scripting language supported by the chosen operating system.
Let’s look at an example of user data.

Placement Groups

Discover Amazon EC2 placement groups and how they help organize instances to improve performance and availability. Learn about cluster, partition, and spread strategies to optimize your application's latency, throughput, and fault tolerance within AWS environments.

When several EC2 instances are launched, AWS offers to distribute the instances physically such that it can reduce the number of system crashes, thus offering high availability. However, this can be troublesome when instances require low-latency communication. Imagine a group of instances that require constant low-latency and high throughput communication to offer services to the users, and each instance is deployed on a different physical structure in an AZ. To cater to such issues, Amazon EC2 also offers placement groups.
A **placement group** allows control of the deployment of interdependent instances across the underlying infrastructure. Placement groups are purposely designed to meet our workload requirements.
Let’s take a deeper look into different placement strategies and their benefits.

Types of placement groups

EC2 offers different placement groups to cater to various application needs related to performance, availability, and fault tolerance. Each placement group offers flexibility to arrange instances as per the needs. Let’s take a look at them in detail.

Cluster placement group

The **cluster placement group** is a logical grouping of interrelated instances to achieve the best throughput and low latency rate possible within an Availability Zone. Instances in the cluster placement group may belong to a single VPC or between peered VPCs in the same region.

Partition placement group

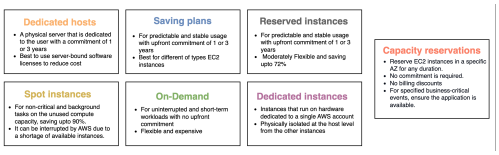
The cluster placement group is designed to launch interconnected instances in close physical proximity to each other on the same infrastructure. The interconnected instances may fail due to correlated hardware failure, thus resulting in application failure. **Partition placement group** helps us further logically divide the placement group into partitions, and each partition has its own rack. Each rack has its network and power source, so no two partitions suffer from correlated hardware issues. Partition placement group also allows to place partitions in different Availability Zones under the same region.

Spread placement group

Partition placement groups still have interconnected instances on the same rack, sharing a power source. That means failure of power in the rack would cause all the instances in that rack to fail. Businesses with an application with a small number of critical instances would like to avoid a placement group with the risk of simultaneous failure. The **Spread placement group** is the placement of instances such that each instance has its rack. It allows each instance on a different rack, therefore suitable for different instance types or launching times.

Types of purchase options

It is important to understand the compute requirements to select the best possible purchase option. The lifecycle of an instance determines the expected EC2 usage patterns (steady/variable, interruptible/not) and the choice of a purchasing option. Amazon EC2 offers multiple purchase options, flexibility, hardware control, and cost efficiency. Some of them are mentioned below:



Amazon Databases services

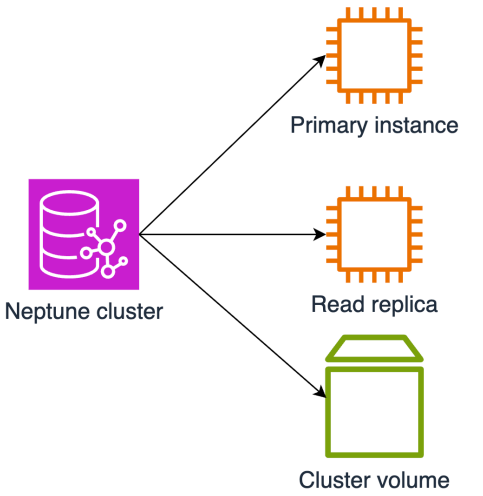
Amazon Neptune

Amazon Neptune

Explore Amazon Neptune, a fully managed graph database service designed for applications with highly connected data. Understand its high performance, scalability, global database capabilities, backup options, and how it supports efficient querying with popular graph models and automatic scaling to meet diverse workloads.

Amazon Neptune is a purpose-built, fast, reliable, and fully managed database service used to run applications with highly connected datasets. Efficient management and rapid querying of interconnected data, such as recommendation patterns, social media activities, and fraud graphs, require a streamlined approach not well-suited to traditional databases.

The main power of Amazon Neptune is its high-performance graph database engine that can store billions of relationships, and querying can be done in milliseconds latency. It supports popular graph models property-graph and W3C's RDF and their respective query languages. Amazon Neptune uses the power of Neptune ML to improve the prediction accuracy by over 50% compared to non-graph databases.
Amazon Neptune is a highly available, durable, and fault-tolerant database. It supports low-latency read replicas across three Availability Zones, making it 99.99% available. It also improves the read capacity and allows it to execute 100k queries per second. A Neptune cluster has three main components: the primary DB instance, replica, and cluster volume. This primary cluster can have a maximum of 15 reader instances.

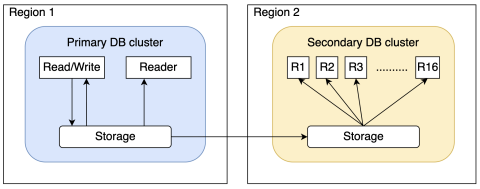


Neptune global database

Amazon Neptune supports global databases to introduce read performance, high availability, and fault tolerance. Global database for Neptune:

- Spans multiple AWS Regions.
- Provides low latency reader instances for global reads.
- Improves fast recovery in case of whole AWS Region goes down.
- Have a primary DB cluster in one AWS Region and can have up to 5 secondary DB clusters in different regions.

Note: Only primary DB cluster can entertain write operations and replicates the data to the secondary DB clusters, and each secondary DB cluster can have up to 16 reader instances.



Performance and scaling

Amazon Neptune enhances the performance by allowing us to scale the resources per our requirements. Scaling is available for the following three resources:

- **Storage scaling:** Neptune automatically scales the cluster's storage and can expand to 128 TiB in all supported AWS Regions. The storage is checked on an hourly basis for cost calculation.
- **Instance scaling:** We can manually scale our DB cluster as required by modifying the existing DB instance class for each instance or adding new instances to the cluster. Each instance can have a different instance class, depending on the workload.
- **Read scaling:** Because Neptune works in a cluster manner, we can add multiple read replicas in a single DB cluster and connect to them directly. All the replicas share the same data as they are attached to the cluster storage.

Amazon Neptune also supports auto-scaling. Once enabled, it monitors the workload, scales out, and scales in automatically to meet our requirements. To enable auto-scaling on a Neptune DB cluster, there must be one primary writer instance and at least one read replica, and no read replica should be in any other state than

“Available.”

Backup and restore

Two types of backups are available in Amazon Neptune:

- **Automated backups:** These are continuous backups created automatically for a specified retention period. The retention period’s default value is 1 day, but it can extend to 35 days. We can’t turn off automated backups on Neptune; all the automated backups are deleted when a DB cluster is deleted.
- **Manual backups:** These are the snapshots of the whole database and are persistent beyond the retention period. These are useful when we want to keep our database longer. Manual backups can be shared across AWS Regions and accounts.

  We can restore the database by creating a new instance from automated or manual backup and modifying the parameters and settings according to the requirements. Amazon Neptune also supports point-in-time recovery from the DB cluster, allowing us to create a new DB instance from any point with 1-second granularity.

Amazon MemoryDB for Redis

Explore Amazon MemoryDB for Redis, an in-memory primary database compatible with Redis that offers ultra-fast read and write performance, multi-availability zone durability, and scalable cluster architecture. Understand its core components, replication mechanisms, scaling options, and backup capabilities to support real-time low-latency applications in retail, finance, and media sectors.

Amazon MemoryDB for Redis is a Redis compatible in-memory primary database that stores the entire datasets in memory across multiple computational nodes. It delivers ultra-fast performance and Multi-AZ durability, allowing microseconds read and single-digit milliseconds write latency. We can build applications quickly by using Redis flexible and friendly data structures and APIs. MemoryDB for Redis stores data using Multi-AZ transactional logs, enabling MemoryDB to deliver Multi-AZ durability for fast database recovery and restart.

Core components

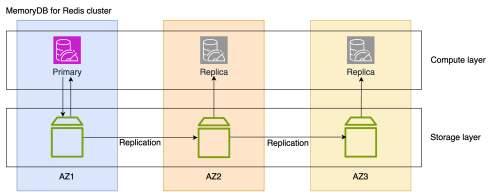
MemoryDB for Redis works in cluster formation. Each cluster consists of shards that contain nodes. Let’s briefly look at these core components of MemoryDB for Redis cluster.

- **Nodes:** A node is an EC2 instance that runs a particular version of the Redis engine selected at the time of cluster creation. It’s the smallest component of a MemoryDB cluster, and we can add or remove nodes from a shard. There are two types of nodes: Primary and Replica. A primary node can entertain read and write requests, while the replica nodes can only have read requests.
- **Shards:** A shard is a collection of one or more nodes. A shard must contain at least one node (primary node) and can have up to five replica nodes, which makes a shard a group of a maximum of six nodes. Our dataset is partitioned across shards.
- **Cluster:** A cluster is a collection of shards responsible for managing the shards and nodes. Each cluster has a specific Redis engine version that defines the functionalities of the cluster. A MemoryDB cluster can have up to 500 shards.

Note: A cluster can have a maximum of 500 shards with no replica nodes or 100 shards with 4 replica nodes totaling 500 nodes per cluster.

Multi-AZ replication and failover

Amazon MemoryDB for Redis cluster spans replica nodes in different availability zones in a region to ensure high data availability. Each replica maintains a copy of the primary node’s storage in their AZ, and the primary node’s data is asynchronously replicated to other nodes using transactional logs. Primary and replica nodes should be spread across multiple AZs in an AWS Region to achieve fault tolerance.
If Multi-AZ is enabled, the primary node failovers to any replica node within a few seconds of downtime.



Scaling

Amazon MemoryDB for Redis allows two ways to scale the resources. One is horizontal scaling, and the second is vertical scaling.
In horizontal scaling, the number of shards is increased or decreased according to the requirement, while in vertical scaling, the capacity of already existing nodes is modified. The scaling process can be offline or online. In offline scaling, the cluster is offline and can’t entertain incoming requests. Therefore, the online scaling is recommended as it allows the cluster to respond to the incoming requests.

Backup and restore

Backup and restore

Amazon MemoryDB takes data backup in Multi-AZ transactional logs to make it highly available across the AWS Region. It also allows us to take periodic or on-demand snapshots for point-in-time recovery. The snapshots are stored in an Amazon S3 bucket, and we can restore it anytime by creating a new cluster from the snapshot.

Note: Amazon MemoryDB does not allow creating more than 20 manual snapshots per cluster within a 24-hour window. Snapshots are available at the cluster level only, and it always taken from the primary instance so that the latest changes are backed up.

Use cases

Here are a few practical use cases where MemoryDB for Redis is beneficial:

- **Retail/e-commerce applications:** Retail customers need speed and agility to keep their applications' loading time as low as possible. A long wait time on a website or difficulty communicating with other services on a website results in lost visitors. MemoryDB for Redis supports applications built on microservices architecture.
- **Financial services:** Financial services are usually time-sensitive, so they require low latencies for efficient data processing and to reflect real-time data; MemoryDB's very low latency helps detect real-time fraud.
- **Media and entertainment:** In the entertainment industry, customers need lighter, faster, and cleaner applications with extreme performance. MemoryDB for Redis can help meet these requirements.

Other Databases

Explore AWS's other database services, including ElastiCache for Redis, DocumentDB, and Keyspaces. Understand their deployment options, use cases, backup features, and how they support scalable and high-performance applications. Learn why QLDB was discontinued and how these services manage data efficiently for various workloads.

In this lesson, we will cover AWS's other database services and their components. Let's start by exploring Amazon ElastiCache for Redis.

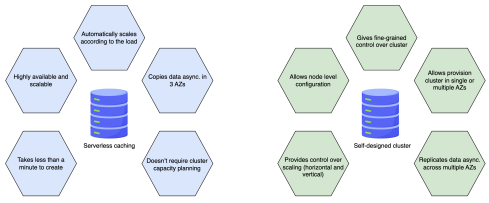
Amazon ElastiCache for Redis

Redis is an in-memory data store that is popular for fast-moving data and real-time applications like leaderboard sessions, chat messaging, and real-time data analytics. However, deploying, monitoring, and scaling are big challenges for Redis applications, and here, the Amazon ElastiCache for Redis comes into play.

Amazon ElastiCache for Redis is a fully managed, high-performance, scalable, and cost-effective service that handles all the management and configuration tasks without our intervention. It is Redis compatible, making it capable of running our existing Redis applications seamlessly. Because, the data is available in the memory, it delivers sub-milliseconds latency and high throughput.

Deployment options

ElastiCache for Redis supports two types of deployments: serverless and self-designed clusters. The serverless option simplifies the deployment and scaling of the cluster according to the application's requirements. In comparison, the self-designed cluster gives more control over the settings and configurations of the cluster. We can provision nodes according to our requirements and enable/disable cluster mode. Enabling cluster mode allows replication across multiple shards to enhance availability and scalability. The diagram below shows a brief comparison between serverless and self-designed clusters:



ElastiCache for Redis enhances performance by caching data in memory, which is ideal for use cases where data resides in another database. In contrast, MemoryDB for Redis is a durable and fast primary database suitable for building applications requiring a high-performance main data store.

Use cases

Here are some of the use cases that can be a good example of Amazon ElastiCache for Redis:

- **E-commerce:** Online stores require maintaining customers' sessions, purchase history, or activity, and this information should be reflected in real-time. There is no need to go to the database for every single request. ElastiCache for Redis can be used to cache the data.
- **Real-time analytics:** We can create business intelligent dashboards to show analytics using real-time data.
- **Gaming:** Leaderboards can be created using ElastiCache for Redis to show the points or standings of players worldwide.

Backup in ElastiCache for Redis

Amazon ElastiCache for Redis supports daily automatic backups, which are retained for 1–35 days and include cluster metadata and cached data. Users can schedule these backups or let ElastiCache set a window. Manual snapshots can be taken and retained indefinitely, with all backups stored in Amazon S3 and available for restoring data by creating a new cluster.

Amazon DocumentDB

Amazon DocumentDB (with MongoDB compatibility) is a fast, scalable, highly available, and fully managed non-relational document database that handles database management tasks like configuring high availability, durability, software patching, scaling, and backing up. It's specially designed for performance gains when working with document workloads. Because of its compatibility with MongoDB, it uses the same commands and tools to insert and query documents.

Components of DocumentDB

Amazon DocumentDB mainly comprises of clusters, instances, and cluster volume.

- **DocumentDB cluster:** A cluster here is a set of instances and storage with instance-based and elastic types. In instance-based clusters, we manage resources, while AWS handles resources automatically in elastic clusters. Clusters can have 0 to 16 instances, with one primary instance handling read/write operations and the rest as read-only replicas.
- **Instance:** An instance is a computation resource with the computation power and memory according to its instance class. Instances should be selected according to application needs. Over-provisioning can lead to unnecessary costs, but the instance type and class can be adjusted.
- **Volume:** Amazon DocumentDB automatically scales cluster volume as the application grows to 128 TiB. All instances in a cluster share this volume, reducing compute scaling time and enabling new read replicas to be ready within 8–10 minutes without data copying. DocumentDB ensures durability by creating six copies of the cluster's volume across three Availability Zones.

Components of DocumentDB

Amazon DocumentDB mainly comprises of clusters, instances, and cluster volume.

- **DocumentDB cluster:** A cluster here is a set of instances and storage with instance-based and elastic types. In instance-based clusters, we manage resources, while AWS handles resources automatically in elastic clusters. Clusters can have 0 to 16 instances, with one primary instance handling read/write operations and the rest as read-only replicas.
- **Instance:** An instance is a computation resource with the computation power and memory according to its instance class. Instances should be selected according to application needs. Over-provisioning can lead to unnecessary costs, but the instance type and class can be adjusted.
- **Volume:** Amazon DocumentDB automatically scales cluster volume as the application grows to 128 TiB. All instances in a cluster share this volume, reducing compute scaling time and enabling new read replicas to be ready within 8–10 minutes without data copying. DocumentDB ensures durability by creating six copies of the cluster's volume across three Availability Zones.

Amazon DocumentDB ensures high availability by allowing replica instances across different AZs, with all instances sharing a six-way replicated storage volume. This design enables rapid read access within 100 milliseconds and allows replicas to be quickly promoted to primary in case of failure, with failover typically completed in 30 seconds.

Backup and restore

Amazon DocumentDB offers two types of backups: continuous and snapshots. Continuous backups are automatically taken throughout the day and have a retention period of 1 to 35 days. They allow point-in-time recovery with a 5-minute RPO. Snapshots are full, permanent backups, either automated or manual, and should be used if you need to retain backups beyond 35 days.

Amazon Keyspaces

Amazon Keyspaces (for Apache Cassandra) is an open-source, NoSQL, scalable, highly available, and fully managed database service. It is a serverless service that saves us from managing and maintaining clusters manually. Keyspaces is used for applications that require fast performance and scalability. It automatically scales up and down the table size per the application's need.

It uses Cassandra Query Language (CQL) and Cassandra drivers, tools, and APIs to run our existing Cassandra applications on AWS. It replicates the table data three times in different AWS Availability Zones to achieve high availability. It helps achieve single-digit millisecond latency irrespective of the size of the database, with 99.99% availability. It is a serverless service, so we don't need to provision computing resources and manage software installations or updates.

Amazon Keyspaces ensures low-latency, highly available, and fault-tolerant data access by allowing global reads and writes across AWS Regions. It uses active-active replication with eventual consistency, resolving conflicts using the last-writer-wins approach.

Use cases for Amazon Keyspaces

Amazon Keyspaces is designed to handle low-latency, mission-critical applications requiring single-digit milliseconds latency. A few of the use cases for Amazon Keyspaces include:

- Migrating Cassandra tables to the cloud to get rid of management and maintenance tasks.
- Building applications like fleet management and route optimization that require low latency.
- Building applications using open-source Cassandra APIs and drivers available for various programming languages.

Amazon Quantum Ledger Database (QLDB)

Amazon Quantum Ledger Database (QLDB) is a purpose-built and fully managed ledger database service that maintains an accurate history of changes made to an application for business and regulatory purposes. It works on blockchain technology, where multiple parties work on the same data. We need to keep track of transactions made by every party and verify the data's authenticity. This data can never be overwritten or deleted, allowing us to backtrack on all the changes made to a transaction.

Amazon QLDB is highly available, replicates the data across 3 AZs, and uses the PartiQL query language. It offers an immutable and cryptographically verifiable ledger database. It is designed to provide a transparent, tamper-evident history of data changes, making it an ideal solution for applications where maintaining an accurate and unalterable record is essential.

Note: As of July 2024, AWS has discontinued QLDB and new customers are not able to create QLDB databases.